

第11章 反射与动态特性

建议学时:3-5 小时 | **难度:**★★★☆☆ | **读者背景:**熟悉 C 的 sizeof、结构体与类型转换;知道 Java/C#/Python 的反射或自省能力(仅作参考,不强求)

🎯 本章学习目标

- 理解反射的含义(运行期类型自省、按名访问、动态创建),并如实认识: Rust 没有反射
- 理解 Rust 拒绝反射的设计取舍:零开销、静态类型安全、无运行时元数据包袱
- 掌握 `std::any::Any`:运行期类型检查与 `downcast` 的有限能力
- 掌握 `TypeId`:类型身份的比较与使用(对应 C 没有、C++ `typeid` 的有限形态)
- 理解 `dyn Trait` 的 `vtable` 是"契约反射":按 `trait` 调用而非按类型自省
- 理解 编译期"反射":`serde` 的 `derive` 宏在编译期读字段并生成代码,替代运行期自省
- 掌握"反射需求"在 Rust 中的替代方案清单:序列化、打印、插件、配置
- 能在设计时判断:这个"反射需求"是真的需要运行期动态,还是可以用编译期解决

💡 从 C 到 Rust:本章主题如何迁移

先说实话:C 没有反射,所以你带着的经验是"没有也能活"。C 里处理"类型未知的数据"靠三样东西:`sizeof`(编译期大小)、`void*` + 手动转回、以及"数据结构的字段顺序即约定"(结构体本质上是手写的字节布局描述)。Java/Python 的反射则是另一极端:运行期能拿到"这个对象的所有字段名和类型、按名调用方法、动态创建实例"——代价是巨大的运行时元数据与性能开销。

Rust 站在 C 一侧,但把工具升级:类型信息仍然主要在编译期,但你需要"运行期知道类型"的地方,有精确的替代品:`Any` 提供"运行期类型检查 + 安全向下转型";`TypeId` 提供类型身份;`dyn Trait` 提供按契约调用;`serde` 宏提供编译期字段遍历(序列化、打印、比较)。本章的路线是"如实承认没有反射,然后逐项看替代品":§11.1-11.2 认识问题,§11.3-11.4 学 `Any/TypeId/dyn`,§11.5-11.6 学编译期方案,§11.7 给一份"反射需求 × Rust 对策"的决策表。

📦 前置知识自查

- C 的 `sizeof`、结构体布局、`void*` 与类型转换;知道"C 没有运行期类型信息"
- 第6章的 `dyn Trait` 与 `vtable`(本章大量复用)
- 第7章的 `serde derive` 用法、第10章的 `derive` 原理
- 第2章的 `match` 与模式匹配(`downcast` 结果用 `match/if let` 处理)
- (可选)Java 的 `getClass/instanceof`、Python 的 `type()/vars()` 用于对照

🚀 本章学习路线

1. **第一阶段(约1小时):**§11.1-§11.2,反射的图景与 Rust 的取舍——先建立正确预期
2. **第二阶段(约1.5小时):**§11.3-§11.4,`Any/TypeId/dyn Trait`——运行期能做什么、不能做什么
3. **第三阶段(约1.5小时):**§11.5-§11.7,编译期替代方案与决策表——生产里的"伪反射"艺术
4. **验收标准:**拿到一个"想用反射"的需求,能五分钟内给出 Rust 替代方案并说出开销差异

本章知识导图

- §11.1 反射是什么:C 没有,Java 有,Rust 呢?
- §11.2 设计取舍:为什么 Rust 拒绝运行期反射
- §11.3 Any:运行期类型检查与安全转型
- §11.4 Typeld 与 dyn Trait:类型身份与契约调用
- §11.5 编译期"反射":serde 的 derive 机制
- §11.6 打印、比较与调试:Debug 与格式化家族
- §11.7 "反射需求"决策表:生产替代方案清单
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§11.1 反射是什么:C 没有,Java 有,Rust 呢?

11.1.1 反射的三层能力

能力	含义	C	Java/C#	Python	Rust
类型自省	运行期间"这个值是什么类型"	无	getClass()	type()	Any + Typeld(有限)
字段/方法遍历	运行期枚举字段名与类型、按名调用	无	getFields()	vars()/getattr()	无(编译期宏替代)
动态创建	运行期按字符串创建实例	无	newInstance()	globals()[name]	无(工厂函数/宏替代)

结论直白:Rust 在"类型自省"上有精确的最小能力(Any),在"字段遍历与动态创建"上没有运行期能力——这些被移到编译期(宏)。这不是缺失,是取舍:运行期反射的元数据与开销,换不来 Rust 场景里需要的东西。

11.1.2 C 的替代方案:约定与手写

```
// C:没有运行期类型信息,程序员自己造
struct Entry {
    int kind;           // 0=int, 1=double, 2=char*
    union {             // 类型自省的"手工版"
        int i;
        double d;
        char *s;
    } val;
};

// 打印"任意 Entry":switch 是手写反射
void print_entry(const struct Entry *e) {
    switch (e->kind) {
        case 0: printf("%d", e->val.i); break;
        case 1: printf("%f", e->val.d); break;
        case 2: printf("%s", e->val.s); break;
    }
}
```

C 的做法是"标签 + 联合体 + switch",全靠程序员维护:kind 写错、新增类型忘改 switch,都是静默错误。Rust 的 enum(第1/3章)把"kind + union + switch"收进一个类型,编译器保证穷尽性——这是"手写反射"的类型安全版。

§11.2 设计取舍:为什么 Rust 拒绝运行期反射

★ 重点:三个理由,对应三条 C 哲学

- **零开销原则**:Rust 承诺"你不用的功能不付钱"。运行期反射要求每个类型携带元数据(字段名表、类型表),连 int 都要能"自省"——这与 C 的"int 就是 4 字节"哲学背道而驰;
- **静态类型安全**:反射的招牌能力"按字符串访问字段"天然绕过类型检查(Java 的 NoSuchFieldException 是运行期错误)。Rust 的目标是把错误留在编译期;
- **可预测的内存与性能**:没有隐藏的元数据分配、没有运行期类型查找的开销,行为可静态推理——服务端与嵌入式场景的硬要求。

代价是实打实的:序列化、调试打印、通用比较需要显式声明(derive 或手写 impl),"按名字段查配置"要多写代码。本章剩余篇幅就是这些代价的补偿方案。

§11.3 Any:运行期类型检查与安全转型

11.3.1 存异构值,查它是什么

```
use std::any::Any;

fn main() {
    // 任何 'static 类型都能放进 Box<dyn Any>(对照 C 的 void*)
    let mixed: Vec<Box<dyn Any>> = vec![
        Box::new(42i32),
        Box::new(String::from("你好")),
        Box::new(3.14f64),
    ];

    for item in &mixed {
        // downcast_ref:若真是该类型则返回 Some(&T),否则 None
        if let Some(n) = item.downcast_ref::<i32>() {
            println!("i32 值:{n}");
        } else if let Some(s) = item.downcast_ref::<String>() {
            println!("String 值:{s}");
        } else if let Some(d) = item.downcast_ref::<f64>() {
            println!("f64 值:{d}");
        } else {
            println!("未知类型");
        }
    }
}
```

对照 C 的 void*: **downcast** 的"猜类型"是带检查的——猜错返回 None,而不是 UB。C 里 void* 转错类型是未定义行为,运行期可能一切正常也可能炸;Rust 的 downcast 把"猜"变成"可失败的检查",安全地失败。

11.3.2 是/不是 检查与值版 downcast

```
use std::any::Any;

fn type_name_of(val: &dyn Any) -> &'static str {
    if val.is::<i32>() {
        "i32"
    } else if val.is::<String>() {
        "String"
    } else if val.is::<f64>() {
        "f64"
    } else {
        "其他"
    }
}

fn main() {
    let a: Box<dyn Any> = Box::new(100i32);
    println!("类型:{}", type_name_of(&*a));

    // 拥有型 downcast:成功返回 Box<T>,失败返回原 Box
    match a.downcast::<i32>() {
        Ok(n) => println!("取回拥有权:{n}"),
        Err(b) => println!("类型不符,原样退回:{:?}", b.type_id()),
    }
}
```

⚠ 易错点 1:Any 只对 'static 类型生效

`&dyn Any` 的约束是 `T: 'static`:含引用的类型(如 `&str`、带生命周期参数的泛型)不能直接 downcast 到具体借用——因为"它是否还活着"与类型系统纠缠。实践中:异构容器装拥有型值(`Box<dyn Any>`),借用型需求用 `dyn Trait`(§11.4)表达。另外注意 `&str` 本身是 'static 的(字面量),可以 downcast;而 `&String` 的引用不可以。

§11.4 TypeId 与 dyn Trait:类型身份与契约调用

11.4.1 TypeId:类型身份的比较

```
use std::any::{Any, TypeId};

// 类型身份:TypeId 可比较、可哈希(对照 C++ 的 typeid)
fn same_type<T: Any, U: Any>() -> bool {
    TypeId::of::<T>() == TypeId::of::<U>()
}

fn main() {
    println!("i32 与 u32 相同?{}", same_type::<i32, u32>()); // false
    println!("i32 与 i32 相同?{}", same_type::<i32, i32>()); // true

    // 动态侧:拿运行期值的 TypeId 与编译期类型比较
    let val: &dyn Any = &7i32;
    if val.type_id() == TypeId::of::<i32>() {
        println!("运行期确认:是 i32");
    }
}
```

11.4.2 dyn Trait 是"契约反射"

```
// 真正生产级的"异构 + 按能力调用":不是"查类型",而是"按契约调用"
// 需要"打印能力"就声明 Print 契约,实现者各写各的
trait Describable {
    fn describe(&self) -> String;
}

struct Server { name: String }
struct Config { port: u16 }

impl Describable for Server {
    fn describe(&self) -> String {
        format!("服务器:{}", self.name)
    }
}

impl Describable for Config {
    fn describe(&self) -> String {
        format!("配置:端口 {}", self.port)
    }
}

fn main() {
    // vtable 里存的是"契约的方法",不是"类型名"
    let items: Vec<Box<dyn Describable>> = vec![
        Box::new(Server { name: "api-01".into() }),
        Box::new(Config { port: 8080 }),
    ];
    for it in &items {
        println!("{}", it.describe()); // 不需要知道具体类型!
    }
}
```

★ 重点:反射 vs 契约,两种思维的差别

反射问的是"你是谁?"(类型名、字段名),契约问的是"你能干什么?"(trait 方法)。Rust 的哲学是后者更符合生产需求:插件、注册表、消息分发——调用方需要的从来不是类型本身,而是某组行为。dyn Trait 的 vtable(第6章)提供行为分发的全部能力,且无需类型名比较。当"行为"能表达需求时,优先 dyn Trait;只有确实需要"按类型分支处理"时才上 Any。

§11.5 编译期"反射":serde 的 derive 机制

11.5.1 序列化是"反射"的头号需求

```
// Java 反射做序列化:运行期遍历字段名/值
// Rust 做法:编译期由宏读取字段,生成序列化代码
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
struct User {
    id: u32,
    name: String,
    tags: Vec<String>,          // 数组
    active: bool,
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let u = User {
        id: 7,
        name: "李明".into(),
        tags: vec!["rust".into(), "server".into()],
        active: true,
    };

    // 序列化:宏生成的代码遍历字段(编译期完成,运行期零元数据)
    let json = serde_json::to_string_pretty(&u)?;
    println!("{}", json);

    // 反序列化:字段名与结构体对齐,类型不符直接报错
    let back: User = serde_json::from_str(&json)?;
    println!("还原:{:?}", back);
    Ok(())
}
```

11.5.2 宏能做"反射"能做的三件事

反射场景	运行期反射(Java)	编译期宏(Rust)
序列化/反序列化	遍历字段名与值	derive(Serialize, Deserialize) 生成代码
调试打印	toString() 反射拼接	derive(Debug) 生成 fmt
深拷贝/比较	反射遍历字段	derive(Clone, PartialEq) 生成实现
字段改名/别名	@SerializedName 注解 + 反射	#[serde(rename = "...")] 属性宏
动态创建	newInstance(类名)	工厂函数 + match,类型安全

宏的"反射"在编译期完成:字段增减时生成代码自动跟随(第10章的 `derive` 原理),运行期零元数据、零动态查找——这是 Rust 对"反射需求"的标准回答:能编译期做的,绝不放运行期。

§11.6 打印、比较与调试:Debug 与格式化家族

11.6.1 Debug 是"结构化自省"的公共接口

```
#[derive(Debug)]
struct Point { x: f64, y: f64 }

fn main() {
    let p = Point { x: 1.0, y: 2.0 };
    // {:?} 是结构化输出:字段名与值,调试日志与错误信息的原料
    println!("{:?}", p);
    // {:#?} 美化多行
    println!("{:#?}", p);
}
```

生产意义:给自定义类型实现 Debug(通常 derive),错误信息与日志就自动携带"完整结构化状态"——这是 C 的 printf("%s", strerror) 永远给不了的现场还原能力。

11.6.2 自定义 Debug 与"脱敏"场景

```
use std::fmt;

// 生产常见:Debug 输出脱敏(密码、令牌不打印明文)
struct Credentials { user: String, password: String }

impl fmt::Debug for Credentials {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        f.debug_struct("Credentials")
            .field("user", &self.user)
            .field("password", &"*****")
            .finish()
    }
}

fn main() {
    let c = Credentials {
        user: "admin".into(),
        password: "s3cret".into(),
    };
    println!("{:?}", c);    // Credentials { user: "admin", password: "*****" }
```

工程建议:错误类型实现 Debug + Display + Error

第9章的 `Box<dyn std::error::Error>` 要求错误实现 Debug 与 Display。徒手写三次样板很烦——生产用 `thiserror` 宏:一行 `#[derive(thiserror::Error, Debug)]` 自动生成(这就是"编译期反射"替代手写样板的标准形态)。规则:自定义错误永远走 `thiserror`,别手写。

§11.7 "反射需求"决策表:生产替代方案清单

需求(常见"想用反射"的场景)	Rust 对策	关键点
打印/调试任意对象	<code>#[derive(Debug)] + {?}</code>	字段自动跟随,脱敏可手写 <code>fmt</code>
序列化到 JSON/TOML/二进制	<code>serde derive</code>	编译期生成,字段改名用属性
深拷贝/比较	<code>derive(Clone, PartialEq)</code>	语义特殊时手写 <code>impl</code>
异构集合按类型处理	<code>enum</code> (穷尽)或 <code>Box<dyn Any> + downcast</code>	类型集合闭合用 <code>enum</code> ,开放用 <code>Any</code>
按行为分发(插件/回调)	<code>dyn Trait / trait object</code>	契约优先于类型名
按字符串选类型/函数	<code>match</code> 字符串到工厂函数	编译期穷尽,运行期无反射查找
数据库 ORM	<code>diesel/sqlx</code> 的 <code>derive</code> 宏	查询结果编译期映射结构体
配置驱动代码生成	过程宏读配置生成代码	生成期在编译期,错误也在编译期

⚠ 易错点 2:把"反射"当第一反应

从 Java/Python 转来的程序员,遇到"对象要能描述自己"第一反应是反射。在 Rust 里第一反应应该是:先问"运行时真的需要动态吗?"。类型集合在编译期就确定的(配置文件字段、协议消息、插件清单),一律 `enum + match + derive`——编译器穷尽检查比任何反射都可靠;只有"未知类型来自运行期"(插件 `dll` 加载、用户输入的类型名)才需要 `Any/dyn`,而且量很小。设计时把这句话贴在桌上:反射是运行期的最后手段,不是第一选项。

⚠ 易错点 3:downcast 分支膨胀是设计信号

如果代码里出现"针对 5 种以上类型各写一个 `downcast` 分支",说明你实际需要的是 `enum`(类型闭合时)或 `trait`(行为分发时)——`Any` 是"未知类型集合"的逃生门,不是常规建模工具。分支数量 = 重构信号:3 种以内还能接受,超过就该回到 §14.3 的 `enum` 建模或 §11.4 的 `dyn Trait`。

本章速查表

主题	结论 / 写法
Rust 有没有反射	没有运行期反射;用 Any/TypeId(自省)与宏(字段遍历)替代
存异构值	Vec<Box<dyn Any>>(对照 void* 数组)
运行期查类型	val.is::<T>() -> bool
安全转型(借用)	val.downcast_ref::<T>() -> Option<&T>,猜错返回 None 而非 UB
安全转型(拥有)	boxed.downcast::<T>() -> Result<Box<T>, Box<dyn Any>>
类型身份	TypeId::of::<T>(),可比较;type_id() 拿运行期值身份
Any 的限制	仅 'static 类型;借用型需求用 dyn Trait
行为分发	dyn Trait + vtable:按契约调用,不查类型名
序列化"反射"	derive(Serialize, Deserialize):编译期生成字段代码
调试打印	derive(Debug) + {:?} / {:#?};脱敏手写 fmt
深拷贝/比较	derive(Clone, Copy, PartialEq)
错误类型样板	thiserror 宏生成 Debug + Display + Error
动态创建	工厂函数 + match(编译期穷尽),而非字符串实例化
选型口诀	编译期能做的绝不放运行期;enum + match 优先,Any/dyn 兜底

最小必会清单

- 能准确说出:Rust 没有运行期反射,但有 Any/TypeId 与编译期宏两大替代
- 能解释 Rust 拒绝反射的三个理由(零开销、静态安全、可预测)
- 能用 Box<dyn Any> 存异构值并 downcast 安全取回
- 知道 downcast_ref 与 void* 转型的本质区别(检查 vs UB)
- 能用 TypeId 比较类型身份,知道 type_id() 的用法
- 能区分"按类型处理"(Any)与"按行为处理"(dyn Trait)并正确选型
- 能解释 serde 的 derive 如何替代反射式序列化,及其开销优势
- 能写出脱敏的 Debug 实现
- 知道 thiserror 宏替代手写错误样板
- 拿到"想用反射"的需求,能套决策表给出 Rust 方案

自测题

Q 1. "Rust 没有反射"这句话准确吗?准确的说法是什么?

✅ 参考答案

不准确(或者说只说对一半)。准确说法:Rust 没有"运行期字段遍历与动态创建"式的反射,但有精确的最小替代——Any 提供运行期类型检查与安全转型(自省的一小部分),TypeId 提供类型身份,宏(serde/derive)在编译期完成"字

段遍历"性质的代码生成。所以:没有"运行期反射",有"编译期反射 + 有限运行期自省"。

Q 2. `void*` 与 `Box<dyn Any>` 都"能装任意类型",生产上为什么差距巨大?

✓ 参考答案

`void*` 取回时"转回正确类型"完全靠程序员记忆与约定,转错是未定义行为(可能静默出错);`Box<dyn Any>` 的 `downcast` 是检查过的转型:类型不符返回 `None` 或 `Err`,安全失败。另外 `Any` 带类型身份(可 `is::<T> / type_id`),而 `void*` 连"它是什么"都无法回答。同样的"异构存储",前者是雷区,后者是可推理的类型。

Q 3. 什么场景用 `downcast` 按类型分支,什么场景用 `dyn Trait` 按行为调用?

✓ 参考答案

判据:你拿到值之后要"按类型分别处理"还是"统一调用某组方法"。前者(事件分发、调试器、反序列化分发)用 `Any + downcast`——处理逻辑以类型为分支;后者(插件、回调、图形绘制)用 `dyn Trait`——调用方只依赖契约,不需要知道类型。能用行为表达需求的,永远优先 `dyn Trait`:它不依赖类型名,扩展新类型不用改分发代码。

Q 4. `serde` 的 `derive` 凭什么替代"反射式序列化"?开销与正确性各如何?

✓ 参考答案

`derive` 宏在编译期读取结构体的字段列表,生成逐字段的序列化/反序列化代码。开销:运行期零元数据、零动态查找,直接执行生成的字段代码(与手写序列化同速);正确性:字段增减自动跟随,类型不符编译期报错,而反射式序列化把"字段不存在/类型错"留到运行期抛异常。代价是结构体必须实现 `serde trait`——而这就是 `derive` 一行的事。

Q 5. "按字符串创建对象"在 `Rust` 里如何实现?为什么比反射更安全?

✓ 参考答案

用 `match` 把字符串映射到工厂函数:`match name { "server" => Box::new(Server::new()), "client" => Box::new(Client::new()), _ => return Err(...) }`。安全性:① 映射关系编译期可见,拼写错误在编译期或测试期暴露;② 每个分支返回的类型有签名约束(如 `Box<dyn Plugin>`),错误提前;③ 不存在的名字是显式 `Err`,而反射的 `NoSuchMethod` 是运行期异常。若类型集合由外部插件提供(运行期才闭合),才需要 `dlopen` + 符号查找——那是少量、受控的 `unsafe` 边界。

Q 6. `Debug` 脱敏怎么写?为什么生产日志里必须做?

✓ 参考答案

手写 `impl fmt::Debug`,用 `debug_struct().field()` 逐个输出字段,敏感字段输出掩码(如 `"*****"`),或用 `#[derive(Debug)]` 后对敏感字段标 `#[debug(skip)]`(需要 `debug` 宏辅助库)。必要性:错误与日志会自动打印 `Debug`,一旦 `Credentials` 混入错误链,明文密码就进了日志文件——这是 C 时代 `print` 结构体字段时同样存在的风险,而 `Rust` 的 `Debug` 契约让"脱敏"有了标准实现点。

章末实验题:通用结构体打印机与异构处理

实验 11:通用结构体打印机

实验目标:写一个"通用打印机":接受任意异构值集合,输出每个值的类型信息与内容;再叠加 serde 序列化输出,对比"运行期自省"与"编译期生成"两条路线。

实验要求:

- 任务 1:定义 3-4 个不同结构体(至少一个含嵌套字段)(覆盖:类型设计)
- 任务 2:用 Box<dyn Any> 集合存放异构值,遍历输出类型名(自定义映射)(覆盖:Any §11.3)
- 任务 3:downcast 到具体类型并打印字段内容,类型不符走兜底(覆盖:downcast §11.3)
- 任务 4:用 TypeId 精确比较并打印"是否匹配"信息(覆盖:TypeId §11.4)
- 任务 5:为结构体 derive(Serialize),把整个集合序列化输出(覆盖:serde §11.5)
- 任务 6:写一个 enum 版"打印机"对比:类型闭合时 enum + match 更优(覆盖:选型 §11.7)

第一步 一步:定义结构体:Server(name, port)、Metric(name, value)、Event(id, kind, msg)

第二步 二步:写 type_label(&dyn Any) 函数,先 is::<T> 链判断返回标签

第三步 三步:写 print_any:downcast 每个类型并打印细节,兜底打印 "未知"

第四步 四步:加入 TypeId 分支输出"类型标识一致"验证

第五步 五步:derive Serialize,用 serde_json 输出整个列表;对比与任务 3 的实现差异

参考答案要点

```
// ===== 实验 11:通用结构体打印机 =====
// 覆盖知识点:Any 与 downcast(§11.3)、TypeId(§11.4)、
//          serde 编译期序列化(§11.5)、enum 选型对照(§11.7)

use std::any::{Any, TypeId};
use serde::Serialize;

// ---- 任务 1:三个结构体 ----
#[derive(Debug, Serialize, Clone)]
struct Server {
    name: String,
    port: u16,
}

#[derive(Debug, Serialize, Clone)]
struct Metric {
    name: String,
    value: f64,
}

#[derive(Debug, Serialize, Clone)]
struct Event {
    id: u32,
    kind: String,
    msg: String,
```

```

}

// ---- 任务 2:类型标签(运行期自省的最小形态) ----
fn type_label(val: &dyn Any) -> &'static str {
    if val.is::<Server>() {
        "Server"
    } else if val.is::<Metric>() {
        "Metric"
    } else if val.is::<Event>() {
        "Event"
    } else {
        "未知"
    }
}

// ---- 任务 3:downcast 打印细节 ----
fn print_any(val: &dyn Any) {
    println!("[{}] ", type_label(val));
    if let Some(s) = val.downcast_ref::<Server>() {
        println!("服务器 {}:{}", s.name, s.port);
    } else if let Some(m) = val.downcast_ref::<Metric>() {
        println!("指标 {} = {:.2}", m.name, m.value);
    } else if let Some(e) = val.downcast_ref::<Event>() {
        println!("事件 #{} {}:{}", e.id, e.kind, e.msg);
    } else {
        println!("(类型未知,无法打印细节)");
    }
}

// ---- 任务 4:TypeId 精确比较 ----
fn is_server(val: &dyn Any) -> bool {
    val.type_id() == TypeId::of::<Server>()
}

// ---- 任务 6:enum 对照版:类型闭合时更优 ----
#[derive(Debug)]
enum Thing {
    S(Server),
    M(Metric),
    E(Event),
}

fn print_thing(t: &Thing) {
    match t {
        Thing::S(s) => println!("enum Server {}:{}", s.name, s.port),
        Thing::M(m) => println!("enum Metric {} = {}", m.name, m.value),
        Thing::E(e) => println!("enum Event #{} {}", e.id, e.msg),
        // 无兜底分支:新增类型时编译器强制你处理 — 反射给不了这个保证
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // 任务 2+3:异构集合 + 运行期自省
    let mixed: Vec<Box<dyn Any>> = vec![
        Box::new(Server { name: "api-01".into(), port: 8080 }),
    ]
}

```

```

Box::new(Metric { name: "cpu", value: 42.5 })),
Box::new(Event { id: 7, kind: "deploy".into(), msg: "发布完成".into() })),
Box::new(42i32),                // 未知类型进列表
];
for item in &mixed {
    print_any(&item);
    if is_server(&item) {
        println!(" (TypeId 确认:是 Server)");
    }
}

// 任务 5:serde 编译期序列化 — 一条路径输出全部,字段自动跟随
let docs: Vec<serde_json::Value> = mixed.iter().map(|item| {
    // 演示:为每个元素生成 "类型 + JSON" 的汇总
    let label = type_label(&item);
    let value = if let Some(s) = item.downcast_ref::<Server>() {
        serde_json::to_value(s).unwrap_or_default()
    } else if let Some(m) = item.downcast_ref::<Metric>() {
        serde_json::to_value(m).unwrap_or_default()
    } else if let Some(e) = item.downcast_ref::<Event>() {
        serde_json::to_value(e).unwrap_or_default()
    } else {
        serde_json::Value::Null
    };
    serde_json::json!({"type": label, "data": value})
}).collect();
println!("JSON 汇总:\n{}", serde_json::to_string_pretty(&docs)?);

// 任务 6:enum 对照 — 类型闭合、编译器穷尽
let things = vec![
    Thing::S(Server { name: "db-01".into(), port: 5432 }),
    Thing::M(Metric { name: "mem", value: 66.6 }),
];
for t in &things {
    print_thing(t);
}
Ok(())
}

```

设计说明:任务 3 与任务 6 的对照是全章的缩影——Any 版的好处是"集合里能混入未知类型",enum 版的好处是"新增类型编译器逼你处理"。生产判断:类型集合由自己掌控且会增长,用 enum(编译期穷尽);集合要容纳"运行期才出现"的类型,用 Any。任务 5 的 serde 路径展示了"编译期反射"的威力:字段改名、嵌套结构变化,序列化自动跟随,而 Any 版的分支代码必须同步修改——这正是 Rust 把"反射"放在编译期的全部理由。

下一章预告:第12章 注解与编译期配置——C 的 #ifdef/_attribute_ 对应 Rust 的属性系统;#[derive]/#[cfg]/#[allow] 与自定义属性宏,编译期开关的正确用法。